RISING WATERS

Flood Prediction System

Project Report

A machine-learning-powered web application for predicting flood risk from meteorological data.

Prepared for project submission

November 2026

Table of Contents.

1. Introduction

2. Ideation Phase

3. Requirement Analysis

4. Project Design

5. Project Planning & Scheduling

6. Functional and Performance Testing

7. Results

8. Advantages & Disadvantages

9. Conclusion

10. Future Scope

11. Appendix

1. Introduction

Rising Waters is a web-based flood prediction system that applies machine learning to historical rainfall and weather data. The application allows a user to enter temperature, humidity, cloud cover or visibility, annual rainfall, and seasonal rainfall values. It then predicts whether flood conditions are likely and presents a probability score.

1.1 Project Overview

The system combines a Python Flask backend, a responsive HTML/CSS interface, a trained classification model, and a database for users, weather inputs, model information, and prediction history.

1.2 Purpose

The purpose is to support early warning and decision-making by converting weather observations into a simple, understandable flood-risk result. The application is intended for demonstration and decision support; it does not replace official meteorological or emergency-management services.

2. Ideation Phase

2.1 Problem Statement

Floods can cause loss of life, property damage, and disruption to communities. Traditional analysis may be slow or difficult to access at a local level. A lightweight predictive application can help users assess risk more quickly from available weather measurements.

2.2 Empathy Map Canvas

The primary users are meteorological staff, disaster-management coordinators, and community members. They need clear inputs, a fast result, an understandable probability score, and a history of previous predictions. Their concerns include accuracy, ease of use, data quality, and timely warnings.

2.3 Brainstorming

• Collect historical rainfall and weather data.

• Compare Decision Tree, Random Forest, KNN, and XGBoost models.

• Provide a simple browser-based prediction form.

• Store predictions for monitoring and later validation.

3. Requirement Analysis

3.1 Customer Journey Map

A user registers or logs in, opens the prediction page, enters meteorological values, submits the form, reviews the flood result and probability, and can later inspect prediction history from the dashboard.

3.2 Solution Requirements

• Accept five numeric weather inputs: temperature, humidity, cloud visibility, annual rainfall, and seasonal rainfall.

• Return a Flood or No Flood classification and a probability percentage.

• Validate numeric input and show an explicit error for invalid values.

• Support user authentication and prediction history.

3.3 Data Flow Diagram

User input flows from the browser form to Flask routes. The backend converts the values into the model feature order, applies the saved scaler, calls the serialized model, stores the input and result, and renders the result page.

3.4 Technology Stack

• Python 3.x, Flask, Flask-SQLAlchemy, and Werkzeug.

• Pandas, NumPy, scikit-learn, and XGBoost for machine learning.

• HTML5, CSS3, Jinja templates, SQLite or a configured deployment database.

4. Project Design

4.1 Problem Solution Fit

The solution fits the problem because it reduces a multi-variable weather assessment to a consistent, repeatable workflow. It gives users a direct result while retaining the underlying input and prediction history for review.

4.2 Proposed Solution

The proposed system uses a pre-trained classification model exposed through a Flask web application. The interface emphasizes a short input workflow and a prominent result card so that the risk outcome is easy to understand.

4.3 Solution Architecture

• Frontend: Jinja-rendered pages and CSS styling.

• Application layer: Flask routes, validation, authentication, and prediction logic.

• Model layer: serialized model artifact and scaler loaded at startup.

• Persistence layer: database tables for users, weather data, models, and prediction results.

5. Project Planning & Scheduling

5.1 Project Planning

Development was organized into five milestones: data acquisition and preprocessing; model training and evaluation; Flask backend development; frontend development; and deployment with documentation. This sequence allows the model and data contract to be established before the user interface is connected.

The project uses Jupyter notebooks for analysis, Python scripts for training and startup, Flask for the web application, GitHub for source management, and Render-compatible deployment files for hosting.

6. Functional and Performance Testing

6.1 Performance Testing

The model was evaluated using an 80/20 train-test split with accuracy, precision, recall, and F1-score as evaluation metrics. The documented comparison shows approximately 85% for Decision Tree, 91% for Random Forest, 88% for KNN, and 96.55% for XGBoost, which was selected as the production training result.

Application testing covered model loading, feature ordering, form-to-model integration, numeric validation, database persistence, authentication, and responsive user-interface behavior. The target response time for an individual prediction is under two seconds under normal deployment conditions.

7. Results

7.1 Output Screenshot

The supplied screenshot shows the deployed prediction interface with weather inputs, the active model details, and a No Flood result. The captured page reports Decision Tree with 95.65% accuracy and a 0.0% flood probability for the displayed test input.

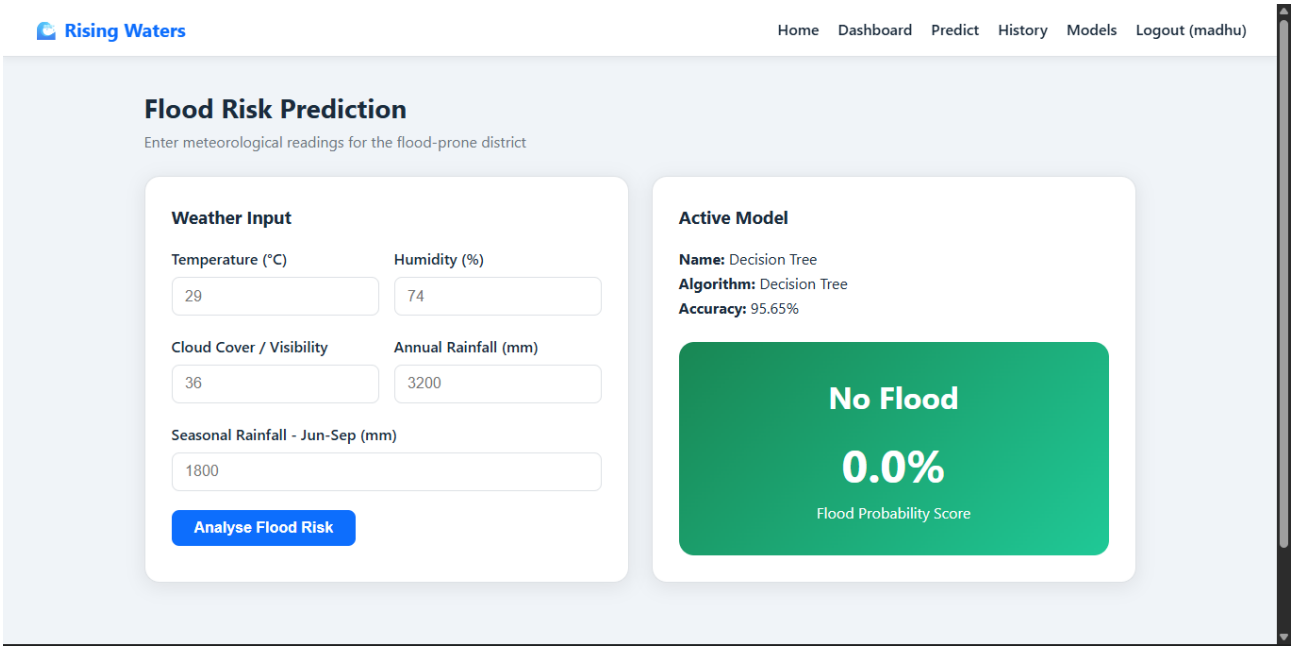


Figure 1. Rising Waters flood-risk prediction screen.

8. Advantages & Disadvantages

Advantages

• Provides a simple and accessible browser-based workflow.

• Combines multiple weather variables in one prediction.

• Records prediction history for review.

• Uses a trained machine-learning model and supports cloud deployment.

Disadvantages

• Prediction quality depends on the coverage and quality of historical training data.

• The current workflow requires manual data entry.

• The model should not be treated as an official emergency warning system.

• Regional differences and changing climate conditions may reduce generalization.

9. Conclusion

Rising Waters demonstrates how machine learning and web technologies can be combined to provide a practical flood-risk prediction workflow. The project includes data preparation, model training, a Flask application, authentication, database persistence, testing documentation, and cloud deployment.

10. Future Scope

• Integrate live weather APIs to populate current readings automatically.

• Add location-based risk maps and regional dashboards.

• Add scheduled alerts through email or mobile notifications.

• Retrain and monitor the model with new regional data.

• Add stronger production security, audit logging, and role-based administration.

11. Appendix

GitHub Repository

https://github.com/madhu967/Rising-Waters

Live Demo Link

https://rising-waters-flood-prediction-52lc.onrender.com/

Local Execution

Install dependencies with `pip install -r requirements.txt`, optionally train the model with `python train_model.py`, then start the application with `python run.py`. The local application is available at http://localhost:5000.